# Задача 0. A + B (не на оценку)

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Во входном файле дано восемь байт, которые задают два 32-битных знаковых целых числа: в первых четырёх байтах записано число $A$, а в последних четырёх — число $B$. Нужно вычислить полусумму этих чисел, округляя полученный результат вниз. Полученную полусумму вывести в выходной файл как 32-битное знаковое целое число. Все три числа заданы с порядком байтов little-endian.

Внимание: В качестве $A$ и $B$ могут быть даны любые числа из диапазона знаковых 32-битных целых чисел (т.е. от INT_MIN до INT_MAX). Настоятельно рекомендуется протестировать решение на числах, близких к крайним значениям, а также на разных комбинациях чётности чисел. Возможно, будет удобнее использовать 64-битные целые для промежуточных результатов, чтобы избежать переполнения.

Весь ввод и вывод в данной задаче бинарный. Ниже показано hex-представление бинарных данных: каждая группа из двух цифр обозначает один байт в файле. Вашей программе на вход будет подан файл с 8 байтами данных, и программа должна создать файл с 4 байтами ответа. Для создания, редактирования и просмотра бинарных файлов используйте какой-нибудь Hex-редактор, например HxD или Hexed.it. Вы также можете установить расширение в VSCode. Пример входных и выходных данных в бинарном виде можно скачать по ссылке.

## Примеры

| input.txt | output.txt |
|---|---|
| AB 05 00 00 12 30 00 00 DE 1A 00 00 | AB 05 00 00 12 30 00 00 DE 1A 00 00 |
| FF FF FF FF FE FF FF FF FE FF FF FF | FF FF FF FF FE FF FF FF FE FF FF FF |
| 0A 0D 0A 0D 0D 0A 0D 0A 8B 8B 8B 0B | 0A 0D 0A 0D 0D 0A 0D 0A 8B 8B 8B 0B |
| 00 00 00 80 00 00 00 80 00 00 00 80 | 00 00 00 80 00 00 00 80 00 00 00 80 |

## Пояснение к примеру

В первом примере даны числа $A = 1451$ и $B = 12306$. Сумма равна 13757, после деления на два получаем 6878.

Во втором примере даны числа $A = -1$ и $B = -2$. Сумма равна $-3$, при делении на два получается $-2$ (округление вниз).

В третьем примере даны большие положительные числа. Если не работает, убедитесь, что открываете файлы в бинарном режиме.

В четвертом примере числа одинаковы и равны INT_MIN = −231. Очевидно, полусумма также равна INT_MIN.

# Задача 1. Сумма чисел

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

В первых четырёх байтах входного файла задано число $N$ — количество чисел в последовательности ($1 \le N \le 10\,000$). Далее заданы сами целые числа последовательности: $N$ штук по 4 байта в каждом. Все числа по абсолютной величине не превышают $10^5$.
Требуется найти сумму $N$ чисел последовательности и вывести её как 4-байтовое целое число.
**Внимание:** Весь ввод и вывод в данной задаче бинарный. В каждом конкретном тесте все числа записаны с одинаковым порядком байтов: это может быть либо big-endian, либо little-endian. Однако в разных тестах порядок байтов может быть разным. Выводить число в ответ требуется с тем же порядком байтов, с которым заданы входные данные.

## Пример

| input.txt |
|---|
| 00 00 00 05 00 00 02 A7 00 00 00 A0 00 00 03 CD 00 00 00 78 00 00 01 B8 |
| output.txt |
| 00 00 09 44 |

## Пояснение к примеру

Учтите, что в примере указано лишь hex-представление бинарных данных! Пример входных и выходных данных можно скачать по ссылке.

# Задача 2. Кодировка UTF-8

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Сегодня повсеместно используется представление символов различных языков при помощи таблицы Unicode. Самая популярная кодировка (т.е. способ сохранения в файл) для Unicode символов — это UTF-8. В данной задаче вам нужно прочитать из входного файла текст в кодировке UTF-8, преобразовать его в последовательность кодов символов, и сохранить все эти коды как 32-битовые целые числа в выходной файл (т.е. по сути записать в кодировке UTF-32LE).

Каждому символу в таблице Unicode назначен код — какое-то неотрицательное целое число. Коды символов таблицы Unicode покрывают весь диапазон от 0x0000 (нуля) до 0x10FFFF включительно, за исключением диапазона от 0xD800 до 0xDFFF, в котором расположены так называемые суррогаты, не соответствующие никаким символам. Таким образом, всего в таблице Unicode 1112064 (0x10F800) символов.

Кодировку UTF-8 можно использовать для представления произвольной последовательности беззнаковых целых чисел (не более 21 бита каждое) в виде последовательности байтов.

Когда в UTF-8 кодируют текст Unicode, то в качестве этих беззнаковых чисел выступают коды символов текста. Чтобы закодировать последовательность значений в UTF-8, нужно представить каждое значение в виде последовательности от одного до четырёх байтов. Далее нужно записать все полученные последовательности байтов подряд друг за другом в порядке записи значений в исходной последовательности. Каждое отдельное значение кодируется согласно таблице:

| длина | шаблон | битов |
|---|---|---|
| 1 | 0xxxxxxx | 7 |
| 2 | 110xxxxx 10xxxxxx | 11 |
| 3 | 1110xxxx 10xxxxxx 10xxxxxx | 16 |
| 4 | 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx | 21 |

Здесь в первом столбце записано количество байтов в представлении, во втором столбце — шаблон, а в третьем — количество букв ‘x’ в шаблоне. В шаблоне записаны в двоичном виде байты, которые получаются при его использовании для представления значения: цифры ‘1’ и ‘0’ обозначают фиксированные значения битов, а буква ‘x’ обозначает бит, состояние которого определяется по кодируемому числу.

Чтобы закодировать целое число $V$ в виде последовательности из $k$ байтов, нужно:

1. Записать число $V$ в двоичной системе исчисления.

2. Найти в таблице $k$-ую строку и зафиксировать её для дальнейших шагов.
3. Если требуется, дополнить $V$ ведущими нулями так, чтобы количество битов в $V$ совпало с числом в последнем столбце таблицы.
4. В буквы ‘x’ шаблона вставить биты числа $V$. Биты вставляются слева направо, в порядке от старшего бита к младшему.
5. В результате вместо шаблона получилось $k$ байтов, по восемь бит в каждом из них — результат кодирования $V$.

Заметим, что если число $V$ в двоичном виде имеет больше битов, чем букв ‘x’ в шаблоне длины $k$, то закодировать его в $k$ байтов нельзя. С другой стороны, для конкретного значения $V$ вполне можно выбрать количество байтов $k$ несколькими способами. Стандарт UTF-8 предписывает выбирать минимально возможное $k$. Если число закодировано в не минимальное количество байтов, то это называется “overlong encoding”, и считается ошибкой.

Например, буква ‘с’ русского алфавита имеет код 1089 в таблице Unicode, что в двоичном виде выглядит как 10001000001. В UTF-8 это число можно записать тремя способами, при этом только первый способ правильный, а все остальные ошибочные:

- 11010001 10000001
- 11100000 10010001 10000001
- 11110000 10000000 10010001 10000001

Во входном файле записан текст в формате UTF-8, возможно с ошибками, общим размером не более мегабайта. Если ошибок нет, то ваша программа должна извлечь коды записанных символов и записать их в выходной файл как 4-байтовые целые числа с little-endian порядком байтов. Если в файле есть ошибки, то они должны быть обработаны строго определённым образом.

Ваша программа должна декодировать символы в том порядке, в котором они записаны в файле. Для каждого символа следует сперва обращать внимание на первый байт и определять по нему количество байтов в представлении символа. Далее нужно считывать остальные байты символа (они называются байтами продолжения) и получать код как целое число. Наконец, нужно проверять, что этот код есть в таблице Unicode и что не имеет место “overlong encoding”.

Есть четыре вида жёстких ошибок:

1. Встретился байт, у которого пять или больше старших битов единичные.
2. Нужно прочитать байт продолжения, чтобы закончить символ, а в файле нет больше байтов.
3. Нужно прочитать байт продолжения, чтобы закончить символ, а следующий байт не имеет формат 10xxxxxx.
4. Первый байт очередного символа имеет формат 10xxxxxx.

При обнаружении жёсткой ошибки программа должна сразу завершаться, не читая остальное содержимое файла. При этом все предыдущие полностью прочитанные символы должны быть в выходном файле.

Кроме того, есть три вида мягких ошибок:

1. Записанный код символа больше 0x10FFFF, а значит не попадает в таблицу Unicode.
2. Записанный код символа попадает в диапазон от 0xD800 до 0xDFFF, то есть является суррогатом.
3. Записанный код символа можно было закодировать меньшим количеством байтов $k$, то есть он закодирован в виде “overlong encoding”.

При обнаружении ошибки код символа следует заменить на 0xFFFD (так называемый “replacement char”), выдать его в выходной файл как обычно и продолжить работу над оставшейся частью файла.

# Примеры

Далее приведены примеры в шестнадцатеричном виде. Скачать их в бинарном виде можно по ссылке. Рекомендуется также смотреть на примеры, записывая байты в двоичном представлении.

| input.txt |
|---|
| 78 E2 89 A4 28 CE B1 2B CE B2 29 C2 B2 CE B3 C2 B2 |
| output.txt |
| 78 00 00 00 64 22 00 00 28 00 00 00 B1 03 00 00 2B 00 00 00<br>B2 03 00 00 29 00 00 00 B2 00 00 00 B3 03 00 00 B2 00 00 00 |
| input.txt |
| 78 F7 89 A4 80 28 CE B1 2B CE B2 29 C2 B2 CE B3 C2 |
| output.txt |
| 78 00 00 00 FD FF 00 00 28 00 00 00 B1 03 00 00 2B 00 00 00<br>B2 03 00 00 29 00 00 00 B2 00 00 00 B3 03 00 00 |
| input.txt |
| 63 6F 6F 6C 20 6D 61 74 68 3A C0 A0 78 E2 89 A4 28 CE B1 2B CE B2 29 C2 B2 CE B3 C2<br>20 78 79 |
| output.txt |
| 63 00 00 00 6F 00 00 00 6F 00 00 00 6C 00 00 00 20 00 00 00<br>6D 00 00 00 61 00 00 00 74 00 00 00 68 00 00 00 3A 00 00 00<br>FD FF 00 00 78 00 00 00 64 22 00 00 28 00 00 00 B1 03 00 00<br>2B 00 00 00 B2 03 00 00 29 00 00 00 B2 00 00 00 B3 03 00 00 |
| input.txt |
| F0 9F 98 82 78 ED A0 90 28 ED B4 80 CE B2 29 FE C2 B2 CE B3 C2 B2 |
| output.txt |
| 02 F6 01 00 78 00 00 00 FD FF 00 00 28 00 00 00 FD FF 00 00 B2 03 00 00 29 00 00 00 |
| input.txt |
| D1 82 D0 B5 D1 81 D1 82 20 85 D1 0D 0A |
| output.txt |
| 42 04 00 00 35 04 00 00 41 04 00 00 42 04 00 00 20 00 00 00 |

# Пояснение к примеру

В первом тесте записан полностью правильный текст из десяти символов. В остальных трёх тестах есть как мягкие, так и жёсткие ошибки.

На втором тесте второй символ имеет код больше 0x10FFFF (мягкая ошибка), так что он заменяется на replacement char. В конце файла записан байт 0xC2, который является началом двухбайтового кода, однако байтов продолжения нет (файл закончился). Это жёсткая ошибка. При этом все символы, прочитанные до этого, сохранены в файл.

На третьем тесте сначала идёт много однобайтовых символов. В середине стоит символ пробела (код 32), записанный двумя байтами. Это overlong encoding, так как можно закодировать его одним байтом 0x20 — заменяется на replacement char (мягкая ошибка). Кроме того, почти в самом конце символ начинается байтом 0xC2, подразумевающим двухбайтовый код, но следующий байт 0x20 не является байтом продолжения — это жёсткая ошибка.

В четвёртом тесте третий и пятый символы имеют коды 0xD810 и 0xDD00 соответственно, которые являются суррогатами (мягкие ошибки). Ближе к концу обнаруживается байт 0xFE, который является жёсткой ошибкой, ибо в UTF-8 байт может иметь максимум четыре старших единичных бита. В пятом тесте после пяти символов внезапно стоит байт продолжения 0x85, хотя на этом месте должен быть первый байт символа — жёсткая ошибка.

# Задача 3. XOR double

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Требуется взять заданное число $X$ типа double, “прохогить” его с заданным 64-битным целым числом $M$, и вывести результат как double. “Прохогить” означает: обратить в битовом представлении $X$ все биты, для которых бит с тем же номером в битовом представлении $M$ равен единице.
В этой задаче предполагается little-endian порядок байтов и общепринятое 8-байтовое представление double.

## Формат входного файла

В первой строке записано целое число $N$ — количество тестов ($1 \le N \le 1\,000$). В остальных $N$ строках записаны тесты, по одному в строке.
Каждый тест описан в формате: "P/Q xor M". Здесь целые числа P и Q — числитель и знаменатель дроби, задающей вещественное число $X$ ($0 \le P \le 100$, $1 \le Q \le 100$), а M — шестнадцатеричное целое число $M$ ровно из шестнадцати цифр. В записи M сначала идут старшие цифры, потом младшие (как обычно у людей записываются числа).
Совет: Шестнадцитеричное число можно читать при помощи формата "%x" так же, как мы считаем десятичные числа форматом "%d". Если нужно прочитать 64-битное число, то нужно дописать две буквы ll перед последней буквой формата.

## Формат выходного файла

Для каждого теста выведите в отдельной строке ($X$ xor $M$) как вещественное число типа double.
Ваши ответы должны быть верны с относительной точностью $10^{-14}$. Рекомендуется использовать формат "%0.15g" при выводе ответа.

# Пример

| input.txt | output.txt |
|---|---|
| 10<br>0/1 xor 8000000000000000<br>1/1 xor 0000000000000000<br>1/1 xor 8000000000000000<br>1/1 xor 3ff0000000000000<br>1/1 xor 7ff0000000000000<br>1/1 xor 0010000000000000<br>1/1 xor 000a000000000000<br>3/7 xor 8000000000000000<br>3/7 xor 8b0abc0000000000<br>3/7 xor 000000000000d000 | -0<br>1<br>-1<br>0<br>2<br>0.5<br>1.625<br>-0.428571428571429<br>-2.90689205178751e-054<br>0.428571428570292 |

# Пояснение к примеру

В первом тесте $X$ = 0/1, а в заданной маске $M$ установлен только старший бит. В представлении нуля в double все биты нулевые, после операции xor старший бит становится единичным, однако число по-прежнему остаётся нулевым (получается так называемый “отрицательный ноль”).

Во втором тесте число $X$ = 1/1 равно единице, а маска $M$ вся нулевая. Значит xor ничего не меняет и результат получается тоже равен единице.

В третьем и восьмом тестах в маске только старший бит единичный. Он в представлении double отвечает за знак числа, так что в этих тестах число $X$ меняет знак.

В предпоследнем тесте число $X$ = 3/7, его битовое представление выглядит как 3fdb6db6db6db6db в шестнадцатеричном виде. Когда мы xor-им с заданной маской, получается представление b4d1d1b6db6db6db. Если проинтерпретировать эти данные как double, то получается число -2.90689205178751e-054.

# Задача 4. Побитовый вывод

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Есть алфавит (набор) из $N$ различных символов, пронумерованных числами от 0 до $N-1$. Есть текст из $M$ символов, все символы которого выбраны из этого набора. Для каждого из $N$ символов задано, какой последовательностью битов он кодируется. Нужно перекодировать весь текст в битовый массив и записать битовый массив в файл.
В данной задаче все числа записаны с little-endian порядком байтов. Более того, при записи битового массива мы считаем, что биты внутри байта также идут в little-endian порядке.

## Формат входного файла

В первых 4 байтах входного файла дано целое число $N$ — количество различных символов в алфавите ($1 \le N \le 10^4$). Далее для каждого из этих символов идёт описание битовой последовательности, на которую его надо заменять. Описание начинается с целого 4-байтового числа $L$ — количества битов в последовательности ($1 \le L \le 64$). Далее идёт $L$ байтов, каждый байт принимает значение 0 или 1 и, по сути, задаёт один бит в последовательности.
Описания битовых последовательностей даны в порядке увеличения номера символа. Биты в каждой последовательности даны в том порядке, в котором их надо записывать в выходной битовый массив.
Далее в файле записано 4-байтовое целое число $M$ — длина текста в символах ($1 \le M \le 10^6$). Наконец, в конце файла записан текст как $M$ символов, каждый записан как 2-байтовое целое число. Гарантируется, что все символы лежат в диапазоне от 0 до $N - 1$.

## Формат выходного файла

Поскольку записать в файл можно только целое количество байтов, битовый массив дополняется минимальным количеством нулевых битов так, чтобы его длина делилась на 8.

# Пример

| input.txt |
| --- |
| 04 00 00 00 0D 00 00 00 01 01 00 01 00 01 01 00<br>00 00 01 01 01 0D 00 00 00 00 01 01 01 01 01 00<br>00 01 01 00 00 01 08 00 00 00 01 01 01 00 01 00<br>00 01 06 00 00 00 01 01 01 01 00 01 08 00 00 00<br>03 00 01 00 00 00 01 00 00 00 01 00 00 00 01 00 |
| output.txt |
| AF CF 5C E3 3E 73 8D FB CC 35 EE 33 01 |

# Пояснение к примеру

Пример в бинарном виде можно скачать по ссылке.

В алфавите 4 символа. Символ 0 заменяется на 1101011000111, символ 1 на 0111110011001, символ 2 на 11101001, а символ 3 на 111101. В тексте 8 символов: 3, 1, 0, 1, 0, 1, 0, 1.

В результате кодирования получается битовый массив: 11110101 11110011 00111010 11000111 01111100 11001110 10110001 11011111 00110011 10101100 01110111 11001100 1. В последней группе только один единичный бит, он дополняется нулями до единичного байта.

# Задача 5. Файлы

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Вася хочет найти кое-какой интересный файл в списке, но, к сожалению, не помнит его имени. Он только помнит, что создал этот файл не раньше, чем в момент времени $A$, и что после момента времени $B$ он точно его не менял. Кроме того, он абсолютно уверен, что этот файл был виден при просмотре. Нужно написать программу, которая будет отбирать файлы, подходящие под этот критерий, и выдавать их Васе.

Список файлов и директорий задан в бинарном виде. Все целые числа записаны с little-endian порядком байтов. Любой момент времени (timestamp) задаётся как количество 100-наносекундных интервалов, прошедших с 1 января 1601 года (текущий момент времени примерно равен 131 832 294 671 670 965). Далее и файлы, и директории мы будет называть обобщённо «файлами».

Входные данные заданы в следующем формате (слева указан диапазон байтов):

| | |
|---|---|
| 0-3 | целое число $N$ — количество файлов в списке ($1 \le N \le 1\,000$) |
| 4-11 | целое число $A$ — искомый файл был создан не раньше этого момента времени ($0 < A < 10^{18}$) |
| 12-19 | целое число $B$ — искомый файл не модифицировался позже этого момента времени ($A < B < 10^{18}$) |
| 20-?? | В оставшейся части входных данных подряд идут описания $N$ файлов. |

Каждый отдельный файл описывается непрерывным блоком данных в формате:

| | |
|---|---|
| 0-20 | Имя файла, состоящее из латинских букв, цифр и символов точки.<br>Длина имени не превышает 20, все оставшиеся байты заполнены нулями |
| 21-28 | Размер $Si$ файла в байтах ($0 \le S_i \le 10^{12}$). |
| 29 | Байт равен 1, если файл на самом деле является директорией, и 0 иначе. |
| 30-37 | Момент времени $Ci$, когда файл был создан ($0 < C_i < 10^{18}$). |
| 38-45 | Момент времени $Mi$, когда файл был изменён в последний раз ($Ci < M_i < 10^{18}$). |
| 46 | Байт равен 1, если файл скрытый, и 0 иначе. |

Гарантируется, что все заданные файлы имеют разные имена. Нужно выбрать среди них файлы, удовлетворяющие критериям:

1. Не директория и не скрытый.
2. Создан не раньше времени $A$, последний раз изменён не позже времени $B$.

Эти файлы нужно отсортировать по имени в лексикографическом порядке, после чего вывести их все подряд в выходные данные. Выводить каждый файл следует блоком данных в том же формате, в котором он был задан во входных данных. Учтите, что при лексикографическом сравнении символы сравниваются по их ASCII-коду.

**Замечание:** Чтобы не писать много кода, можно читать и писать файл целиком как структуру.

# Пример

Пример входных и выходных данных можно скачать по ссылке.

# Задача 6. Простой BSON

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Формат BSON является бинарным вариантом известного формата JSON. Его спецификация доступна по адресу.

В файле input.txt записан документ (объект) в формате BSON, внутри которого нет поддокументов (подобъектов и подмассивов). В этом объекте могут быть лишь поля следующих типов: double, string, bool, null, int32, int64. Нужно прочитать этот BSON, и вывести его в текстовом виде как JSON.

В описании формата BSON используются следующие базовые типы:

- byte: просто байт.
- int32: знаковое 32-битное целое число.
- int64: знаковое 64-битное целое число.
- double: вещественное число двойной точности.

Все числа записываются с little-endian порядком байтов. Целые числа записываются в дополнительном коде, а вещественные — согласно стандарту IEEE 754.

Документ BSON начинается с числа типа int32, которое обозначает размер всего документа в байтах. Далее идёт описание всех полей документа одно за другим. В самом конце документа стоит дополнительный нулевой байт.

Описание поля начинается с одного байта $T$, который определяет, какого типа значение в нём записано:

- $T = 1$: значение типа double.
- $T = 2$: значение типа string.
- $T = 8$: значение типа bool.
- $T = 10$: значение типа null.
- $T = 16$: значение типа int32.
- $T = 18$: значение типа int64.

Сразу после типа записано имя поля в виде строки, заканчивающейся на дополнительный нулевой байт. После этого нулевого байта записано значение поля.

Значение поля типа bool может быть равно либо false, либо true. Значение false записывается в BSON-файле как байт с нулевым значением, а true — как байт с единичным.

Для поля типа null никаких байтов в значение поля не пишется. Значение поля типа string записывается следующий образом. Сначала записано число $L > 0$ типа int32, которое равно количеству байтов в строке, включая дополнительный нулевой байт. В следующих $L-1$ байтах задана сама строка. В конце добавлен дополнительный нулевой байт.

При записи документа в текстовом формате JSON в первой строке пишется открывающая фигурная скобка, а в последней — закрывающая фигурная скобка. Каждая строка между

ними описывает одно поле. Поля нужно описывать в том же порядке, в котором они даны в BSON-файле. Описание поля начинается с имени поля, заключённого в двойные кавычки, и двоеточия сразу после него. Далее должен быть поставлен один пробел, после которого записано значение поля. В самом конце строки должна стоят запятая, если только это поле не является последним полем документа.
Значение типа string заключается в двойные кавычки, остальные значения — нет. Значение типа bool пишется как одно из слов false или true. Значение типа null пишется как слово null. Значение целочисленного типа пишется в десятичной системе исчисления без ведущих нулей (как обычно). Значение типа double должно быть записано при помощи printf с форматом "%0.15g".
Гарантируется, что все числа типа double являются нормальными числами и по модулю не превышают 10100. Гарантируется, что все строковые значения и имена полей не содержат символов, требующих экранирования в формате JSON. То есть они не содержат символов обратного слэша (ASCII 92), двойных кавычек (ASCII 34), а также контрольных символов (ASCII 0-31). Учтите, что строковые значения и имена полей задаются в кодировке UTF-8, что, в частности, имеет значение для русских символов в примере.
Размер входного файла не превышает один килобайт.

# Пример

| input.txt |
|---|
| 5F 00 00 00 02 48 57 00 0E 00 00 00 48 65 6C 6C<br>6F 2C 20 57 6F 72 6C 64 21 00 02 D0 9F 20 D0 9C<br>00 17 00 00 00 D0 9F D1 80 D0 B8 D0 B2 D0 B5 D1<br>82 20 D0 B2 D1 81 D0 B5 D0 BC 21 00 01 64 62 6C<br>00 66 66 66 66 66 BA 81 40 10 69 6E 74 00 25 00<br>00 00 0A 6E 6F 6E 65 00 08 62 6C 6C 00 01 00 |
| output.txt |
| {<br>"HW": "Hello, World!",<br>"П М": "Привет всем!",<br>"dbl": 567.3,<br>"int": 37,<br>"none": null,<br>"bll": true<br>} |

Пример входных и выходных данных можно скачать по ссылке.

В целях тестирования вы можете создавать BSON-файлы при помощи online-конвертера. Учтите, что конвертация обратно в JSON на этом сайте не работает.

Кроме того, вы можете использовать для конвертации следующие скрипты на языке Python (не забудьте запустить “pip install bson” после установки Python 3):

```
import bson
import json
with open("input.txt", "rt", encoding="utf-8") as f:
```

```
    data = json.load(f)
with open("output.txt", "wb") as f:
    f.write(bson.dumps(data))
```

---

```
with open("input.txt ", "rb") as f:
    data = bson.loads(f. read())
with open("output.txt", "wt", encoding="utf-8") as f:
    json.dump(data, f, indent=4, ensure_ascii=False)
    print("", file=f
```

# Задача 7. Пересечение множеств

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Дан набор из $N$ различных битовых масок. Две битовые маски называются конфликтующими, если у них есть общий единичный бит (хотя бы один). Иными словами, если $k$-ый бит равен единице в обеих масках для некоторого $k$, тогда маски конфликтуют. Требуется определить, сколькими способами можно выбрать пару неконфликтующих масок из заданного набора.

## Формат входного файла

В первой строке входного файла задано целое число $N$ — количество масок ($1 \le N \le 3 \cdot 10^4$). В каждой из оставшихся $N$ строк записано по одной битовой маске. Битовая маска — это беззнаковое 64-битное число, заданное в шестнадцатеричном виде ровно шестнадцатью цифрами.
Для чтения битовых масок рекомендуется использовать формат: %llx.

## Формат выходного файла

Выведите одно целое число (в десятичном виде) — количество пар неконфликтующих масок.

## Пример

| input.txt | output.txt |
|---|---|
| 7<br>0000000000000000<br>FFFFFFFFFFFFFFFF<br>0000100000000000<br>0000000004000000<br>0000A00000000000<br>0000666666600000<br>0000000012345000 | 13 |

## Комментарий

Чтобы решение работало быстро, нужно определять наличие конфликта очень быстро, не перебирая отдельные биты в масках.

# Задача 8. TGA в BMP

| | |
|---|---|
| Источник: | повышенной сложности |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

В данной задаче требуется прочитать из файла растровое изображение в формате Targa (Truevision TGA) и записать это изображение в выходной файл в формате BMP.

О формате TGA можно прочитать в википедии. Более точно описание формате приведено здесь. В данной задаче про входной TGA гарантируется, что:

1. Байт 1 "Color Map Type" равен нулю, то есть файл не содержит карты цветов.
2. Байт 2 "Image Type" равен 2 или 10, то есть в файле хранится TrueColor изображение, причём возможно с RLE-сжатием, а возможно без него.
3. Байт 16 "Image Pixel Size" равен 24 или 32, то есть закодированы красная, зелёная и синяя компоненты, а иногда ещё и альфа-канал.
4. Файл может быть записан в новом формате, т.е. содержать подпись XTRUEVISION-XFILE в конце. Однако даже если это так, то следует полностью игнорировать опциональные области: "Developer Area", "Extension Area", "TGA File Footer".
5. Ширина и высота изображения положительные и не превышают 2048.

О формате BMP можно прочитать в википедии. На выходной файл накладываются следующие дополнительные требования:

1. Структура с информационными полями "DIB header" должна быть самой старой версии CORE и иметь размер 12 байт.
2. Поле bcBitCount должно быть равно 24, то есть пиксели должны задаваться в формате RGB (если был альфа-канал, то его надо выбросить).
3. Пиксельные данные должны начинаться сразу после всех хедеров, то есть поле bfOffBits должно быть минимально возможным.

С учётом этих дополнительных правил существует единственный вариант выходного файла. Выданный вашим решением BMP-файл будет засчитан, только если он полностью побайтово совпадает с правильным ответом.

## Пример

Пример входных и выходных данных можно скачать по ссылке.

# Задача 9. Битовый массив

| | |
|---|---|
| Источник: | повышенной сложности |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | 8 мегабайт |

Память современных компьютеров можно представить в виде очень длинного массива битов, в котором каждый бит может принимать значения 0 или 1. Однако возможность обращаться к конкретному биту по его номеру отсутствует: минимальная адресуемая единица памяти — это байт, обычно состоящий из 8 битов. Чтобы работать с отдельными битами памяти, необходимо обращаться к байтам (или словам), которые их содержат, и применять различные битовые операции.

В данной задаче нужно реализовать структуру данных “битовый массив” / “bitset”. Это массив, элементы которого принимают лишь значения 0 и 1. Эти элементы можно легко сохранить в массиве байтов, храня одно значение в каждом байте. Однако если сохранять по 8 элементов в байте, то снижается потребление памяти и появляется возможность ускорить некоторые операции — именно это и требуется сделать в данной задаче.

Нужно реализовать набор функций со следующей сигнатурой:

```
// какой-нибудь целочисленный тип (желательно беззнаковый)
typedef ??? bitword;
// инициализирует массив битов длины num, устанавливая все его биты в ноль
void bitsetZero(bitword *arr, int num);
// возвращает значение idx-ого бита (0 или 1)
int bitsetGet(const bitword *arr, int idx);
// устанавливает значение idx-ого бита в newval (которое равно 0 или 1)
void bitsetSet(bitword *arr, int idx, int newval);
// возвращает 1, если среди битов с номерами k
// для left <= k < right есть единичный, и 0 иначе
int bitsetAny(const bitword *arr, int left, int right);
// установить в val значение всех k-ых битов для left <= k < right
void bitsetSetSeg(bitword *arr, int left, int right, int newval);
// посчитать, сколько битов равно единице на отрезке left <= k < right
int bitsetCount(const bitword *arr, int left, int right);
```

Массив слов arr управляется вызывающим, и может указывать, к примеру, на глобальный массив достаточного размера. Вы можете самостоятельно выбрать базовый тип для слова bitword. Однобайтовый тип подойдёт, однако для лучшей производительности рекомендуется использовать слова большего размера. Функция bitsetAny определяет, есть ли в заданном окне/отрезке битов хотя бы один бит, равный единице. При реализации этой функции необходимо быстро обрабатывать окна большого размера: для этого нужно целиком перебирать слова, полностью попадающие внутрь окна, не перебирая отдельные биты.

При помощи реализованной структуры нужно решить тестовую задачу.

**Внимание:** в этой задаче ограничение по времени выставлено очень сурово!

# Формат входного файла

В первой строке записано целое число $N$ — количество операций, которые нужно обработать ($1 \le N \le 2 \cdot 10^5$). В каждой из следующих $N$ строк описывается одна операция над битовым массивом.

Описание операции начинается с целого числа $t$, обозначающего тип операции. Если $t = 0$, то это операция bitsetZero, и вторым целым числом в строке указан размер массива num. Если $t = 1$, то это операция bitsetGet, и в строке также записано целое число idx — номер бита. Значение этого бита нужно выдать в выходной файл. Если $t = 2$, то это операция bitsetSet, и в строке также содержатся целые числа idx и newval. Здесь idx — номер бита, который нужно изменить, а newval — новое значение, которое надо записать (0 или 1). Если $t = 3$, то это операция bitsetSetSeg, и указано ещё три параметра left, right и newval — отрезок, на котором нужно выполнить присваивание, и новое значение для битов этого отрезка. Если $t = 4$, то это операция bitsetCount, и указано ещё два параметра left и right — отрезок, на котором нужно посчитать количество единичных битов. Полученный результат нужно выдать в выходной файл.

Размер массива num больше нуля и не превышает $2 \cdot 10^7$. Гарантируется, что после операции bitsetZero с параметром num все последующие операции обращаются только к битам с номерами от 0 до num-1 включительно — как минимум до следующего вызова bitsetZero.

Сумма значений num по всем операциям bitsetZero не превышает $10^{10}$. Сумма длин отрезков (right-left) по всем операциям bitsetSetSeg не превышает $10^{10}$, аналогично для операций bitsetCount.

# Формат выходного файла

Для каждой операции bitsetGet или bitsetCount нужно вывести ответ в отдельной строке.

# Пример

| input.txt | output.txt |
|---|---|
| 14<br>0 100<br>2 30 1<br>2 31 1<br>2 32 1<br>1 31<br>1 7<br>2 31 0<br>1 31<br>3 30 33<br>3 31 32<br>3 0 100<br>3 45 67<br>0 48<br>1 30 | 1<br>0<br>0<br>some<br>none<br>some<br>none<br>0 |
| 10<br>0 100<br>3 17 54 1<br>4 10 25<br>1 17<br>1 54<br>1 32<br>2 31 0<br>3 15 20 0<br>4 10 20<br>4 19 59 | 8<br>1<br>0<br>1<br>0<br>33 |

# Пояснение к 1 примеру

Сначала инициализируется массив из 100 битов. Потом устанавливаются в 1 биты с номерами 30, 31, 32. Потом делается запрос на значения битов 31 и 7 — они равны 1 и 0 соответственно. Далее 31-ый бит зануляется и запрашивается его уже нулевое значение. Потом делаются запросы о том, есть ли единичные биты на отрезках [30, 33), [31, 32) и [0, 100).

***Обратите внимание***, что правый конец отрезка в данной задаче исключается. Наконец, выполняется пересоздание массива, теперь размера 48 битов, в результате 30-ый бит становится нулевым.

# Комментарий

Вычисление количества единичных битов в целом числе — весьма интересная задача сама по себе. Для решения этой задачи вам потребуется какое-нибудь быстрое решение. Много хороших вариантов можно найти в bit twiddling hacks (эта страница вообще очень познавательная) или в этом вопросе на SO.